

DeepPose (Colab) — fully commented, step-by-step code

Below is your Colab notebook code rewritten cleanly with comments at every step. I've fixed minor formatting/typos from the PDF export (like stray spaces and line breaks) so it's copy-paste ready in Colab.

Environment setup

1) Check Python version (helps with debugging environment issues)

```
import sys
```

```
print(sys.version)
```

Upgrade pip to ensure latest packages install cleanly

```
!python -m pip install --upgrade pip
```

2) Install all necessary libraries for the project

- opencv-python: camera/image processing

- mediapipe: human pose landmarks

- torch, torchvision, torchaudio: deep learning (PyTorch + CV helpers)

- numpy, pandas: numerical/data utils (pandas optional)

- scikit-learn: metrics like classification report, confusion matrix

- matplotlib, seaborn: plotting training curves

- tqdm: progress bars for training

- Pillow: image handling (PIL)

```
!pip install opencv-python mediapipe torch torchvision torchaudio numpy pandas scikit-learn matplotlib seaborn tqdm Pillow
```

Drive mounting and dataset extraction

Mount your Google Drive so you can read/write files (datasets & checkpoints)

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

Unzip your dataset from Drive into the Colab runtime (fast local access)

```
import zipfile
```

```
from pathlib import Path
```

```
zip_path = "/content/drive/MyDrive/Colab_Notebooks/DeepPose_Datasets.zip" # path in Drive
```

```
extract_path = "/content/Dataset" # local path inside Colab VM
```

Create the extraction folder if it doesn't exist

```
Path(extract_path).mkdir(parents=True, exist_ok=True)
```

Extract the entire zip to /content/Dataset

```
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
```

```
    zip_ref.extractall(extract_path)
```

Optional: list the extracted directory to verify structure

```
!ls -R /content/Dataset
```

Define train/test directory paths as used by ImageFolder

NOTE: Adjust these if your extracted zip uses different names/case.

```
train_dir = "/content/Dataset/DATASET/TRAIN"
test_dir = "/content/Dataset/DATASET/TEST"
```

Basic existence checks for sanity

```
import os
print("Train exists:", os.path.exists(train_dir))
print("Test exists:", os.path.exists(test_dir))
```

Datasets and transforms

Avoid errors with partially saved images; allows PIL to load truncated files

```
from PIL import ImageFile
ImageFile.LOAD_TRUNCATED_IMAGES = True
```

Torchvision dataset/transforms and DataLoader

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

Use ImageNet normalization because we will fine-tune a model pretrained on ImageNet

```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]
```

Training transforms:

- Resize to 224x224 (expected by ResNet-18)

- Data augmentation (rotation, flip, color jitter) to reduce overfitting

- Convert to tensor and normalize with ImageNet stats

```
transform_train = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomRotation(20),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.1, hue=0.05),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])
```

Test/Validation transforms:

- Deterministic (no augmentation)

- Same resize/normalize as training

```
transform_test = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])
```

Build dataset from folders structured as:

train_dir/<class_name>/image.jpg

test_dir/<class_name>/image.jpg

```
train_dataset = datasets.ImageFolder(root=train_dir, transform=transform_train)
test_dataset = datasets.ImageFolder(root=test_dir, transform=transform_test)
```

Create loaders:

- Training loader shuffles data

- Test loader is deterministic

```
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, num_workers=2,
pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, num_workers=2,
pin_memory=True)
```

Inspect classes and dataset sizes

```
classes = train_dataset.classes
print("Classes:", classes)
print("Train size:", len(train_dataset))
print("Test size:", len(test_dataset))
```

Model setup (ResNet-18 transfer learning)

```
import torch
import torch.nn as nn
import torchvision.models as models
from torchvision.models import ResNet18_Weights
```

Pick CUDA if available, else CPU

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_classes = len(classes)
```

Load ResNet-18 pretrained on ImageNet

```
model = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
```

Freeze backbone (all layers except the final fully-connected "fc")

This speeds up training and avoids overfitting with small datasets.

```
for name, p in model.named_parameters():
    if not name.startswith("fc."):
        p.requires_grad = False
```

Replace final FC layer to match the number of pose classes

```
in_feat = model.fc.in_features
model.fc = nn.Linear(in_feat, num_classes)
```

Move model to device

```
model = model.to(device)
```

Loss function: multi-class classification

```
criterion = nn.CrossEntropyLoss()
```

Optimizer on the trainable parameters (only the final FC here)

Small weight_decay adds regularization

```
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3, weight_decay=1e-4)
```

Training and evaluation loops

```
from tqdm import tqdm
import numpy as np
```

```
EPOCHS = 25      # total training epochs  
best_val_acc = 0.0 # to track best validation accuracy
```

```
# Lists to store training/validation loss/accuracy for plotting later
```

```
train_losses, val_losses = [], []  
train_accs, val_accs = [], []
```

```
# Helper: evaluate model on a given loader (returns avg loss and accuracy)
```

```
def evaluate(loader):  
    model.eval()  
    running_loss, correct, total = 0.0, 0, 0  
    with torch.no_grad():  
        for images, labels in loader:  
            images, labels = images.to(device), labels.to(device)  
            logits = model(images)  
            loss = criterion(logits, labels)  
            preds = logits.argmax(1)  
  
            running_loss += loss.item() * images.size(0)  
            correct += (preds == labels).sum().item()  
            total += images.size(0)  
  
    return running_loss / total, correct / total
```

```
# Main training loop
```

```
for epoch in range(1, EPOCHS + 1):  
    model.train()  
    running_loss, correct, total = 0.0, 0, 0
```

```
# Iterate over training batches with a progress bar
```

```
for images, labels in tqdm(train_loader, desc=f"Epoch {epoch}/{EPOCHS}"):   
    images, labels = images.to(device), labels.to(device)
```

```
optimizer.zero_grad()      # reset gradients  
logits = model(images)      # forward pass  
loss = criterion(logits, labels) # compute loss  
loss.backward()             # backpropagate  
optimizer.step()            # update weights
```

```
preds = logits.argmax(1)    # predictions for accuracy  
running_loss += loss.item() * images.size(0)  
correct += (preds == labels).sum().item()  
total += images.size(0)
```

```
# Compute epoch training metrics
```

```
t_loss = running_loss / total  
t_acc = correct / total
```

```
# Evaluate on validation/test set
```

```
v_loss, v_acc = evaluate(test_loader)
```

```
# Save metrics for plots
```

```

train_losses.append(t_loss); train_accs.append(t_acc)
val_losses.append(v_loss); val_accs.append(v_acc)

print(f"Epoch {epoch}/{EPOCHS} | Train: loss={t_loss:.4f}, acc={t_acc:.3f} | Val: loss={v_loss:.4f},
acc={v_acc:.3f}")

# Save the best checkpoint based on validation accuracy
if v_acc > best_val_acc:
    best_val_acc = v_acc
    torch.save({
        "model_state": model.state_dict(),
        "classes": classes
    }, "/content/resnet18_yoga_best.pth")
    print(f"Saved best checkpoint: /content/resnet18_yoga_best.pth (val_acc={v_acc:.3f})")

```

Training curves (loss and accuracy plots)

```

import matplotlib.pyplot as plt

plt.figure(figsize=(10, 4))

# Loss curves
plt.subplot(1, 2, 1)
plt.plot(train_losses, label="Train")
plt.plot(val_losses, label="Val")
plt.title("Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

# Accuracy curves
plt.subplot(1, 2, 2)
plt.plot(train_accs, label="Train")
plt.plot(val_accs, label="Val")
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.tight_layout()
plt.savefig("/content/training_curves.png", dpi=150) # save to Colab VM
plt.show()

```

Classification report and confusion matrix

```

import numpy as np
from sklearn.metrics import classification_report, confusion_matrix

# Collect predictions on the test set to compute detailed metrics
model.eval()
y_true, y_pred = [], []

```

```
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        logits = model(images)
        preds = logits.argmax(1).cpu().numpy() # predicted class indices
```

```
y_true.extend(labels.cpu().numpy())
y_pred.extend(preds)
```

Print per-class precision/recall/F1 and overall metrics

```
print("Classification Report:")
print(classification_report(y_true, y_pred, target_names=classes))
```

Print confusion matrix to see which classes get mixed up

```
print("Confusion Matrix:")
print(confusion_matrix(y_true, y_pred))
```

Save artifacts to Drive

Copy best checkpoint and training curves image to your Drive for persistence

```
!cp /content/resnet18_yoga_best.pth /content/drive/MyDrive/
!cp /content/training_curves.png /content/drive/MyDrive/
print("Copied checkpoint and curves to Drive.")
```

Single-image inference (quick sanity check)

Simple interactive test: upload one image and run model inference

```
from google.colab import files
from PIL import Image
import torchvision.transforms as T
import torch.nn.functional as F
```

Prompt user to upload an image from local files

```
uploaded = files.upload() # choose one yoga image file
img_name = list(uploaded.keys())[0]
```

Inference transform (must match resize/normalize used elsewhere)

```
infer_tf = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])
```

Load image, convert to RGB, and prepare tensor batch

```
img = Image.open(img_name).convert("RGB")
x = infer_tf(img).unsqueeze(0).to(device)
```

Run model to get logits and softmax probabilities

```
model.eval()
with torch.no_grad():
```

```
logits = model(x)
probs = F.softmax(logits, dim=1)
conf, pred_idx = torch.max(probs, dim=1)
```

```
print(f"Predicted pose: {classes[pred_idx.item()]} | Confidence: {conf.item():.2f}")
```

What each part does (quick recap)

- **Environment setup:** ensures all dependencies are installed and pip is up to date.
- **Drive + unzip:** brings your dataset into Colab's local filesystem for faster training.
- **Transforms:** standardize images and add augmentation for robustness.
- **ImageFolder + DataLoader:** turn your folder dataset into batches.
- **ResNet-18 + freeze:** leverage pretrained features; train only the final layer for your classes.
- **Training loop:** iterate over data, compute loss/accuracy, and update weights.
- **Evaluation:** measure performance on test/validation data each epoch.
- **Checkpointing:** save the best model to reuse in your local demo (main.py).
- **Metrics:** classification report and confusion matrix to diagnose weak classes.
- **Artifacts to Drive:** persist your best weights and training curves.
- **Single-image test:** quick verification that the checkpoint predicts sensibly.

Config.py

Why this file matters

- Keeps all constants/configs in one place (paths, classes, thresholds).
- Makes your code cleaner: instead of hardcoding values in train.py or main.py, you import from config.py.
- Easy to update later (e.g., add new poses, change confidence threshold).

--- Import required libraries ---

```
import torch          # PyTorch for device selection (CPU/GPU)
from pathlib import Path # Pathlib for handling filesystem paths in a clean way
```

--- Directory paths ---

```
CKPT_DIR = Path("checkpoints") # Folder where trained model checkpoints (.pth files) are saved
LOG_DIR = Path("logs")         # Folder where training logs or outputs can be stored
```

--- Class labels ---

```
CLASSES = ["downdog", "goddess", "plank", "tree", "warrior"]
```

List of yoga poses (the categories your model will classify).

Must match the folder names in your dataset and the order used during training.

--- Device configuration ---

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Automatically selects GPU if available, otherwise falls back to CPU.

This ensures your code runs on both Colab (GPU) and local machine.

--- Inference settings ---

```
CONFIDENCE_THRESHOLD = 0.75 # Minimum probability required to accept a prediction as "confident"
```

```
SMOOTHING_WINDOW = 5 # Number of frames to average predictions over (reduces flickering in real-time demo)
```

--- Normalization constants ---

MEAN = [0.485, 0.456, 0.406] **# ImageNet mean values for RGB channels**

STD = [0.229, 0.224, 0.225] **# ImageNet standard deviation values**

These are used to normalize images before feeding them into ResNet-18,

ensuring consistency with how the model was pretrained.

Main.py

Key things to remember

- Model loading: Restores ResNet-18 with your trained weights.
- Preprocessing: Must match training normalization.
- Prediction: Uses softmax + confidence threshold.
- MediaPipe: Provides landmarks for feedback rules.
- Feedback: Simple angle checks for Warrior, Goddess, etc.
- Smoothing: Avoids flickering predictions by averaging last few frames.
- Loop: Reads frames, predicts, overlays text, shows window, exits on q.

--- Import required libraries ---

import cv2, torch, torch.nn as nn, numpy as np

from torchvision import transforms, models

from torchvision.models import ResNet18_Weights

from collections import deque

import mediapipe as mp

from config import * **# Import constants/configs (paths, classes, thresholds, normalization) from config.py**

--- Load trained model checkpoint ---

ckpt = torch.load(CKPT_DIR / "resnet18_yoga_best.pth", map_location=DEVICE) **# Load saved weights**

model = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1) **# Base ResNet-18**

pretrained on ImageNet

model.fc = nn.Linear(model.fc.in_features, len(ckpt["classes"])) **# Replace final layer with correct #classes**

model.load_state_dict(ckpt["model_state"]) **# Load trained weights**

model.to(DEVICE).eval() **# Move to device (CPU/GPU) and set eval mode**

classes = ckpt["classes"] **# Class names list**

--- Preprocessing transforms (must match training) ---

```
val_tf = transforms.Compose([
    transforms.Resize((224, 224)), # Resize to ResNet input size
    transforms.ToTensor(), # Convert to tensor
    transforms.Normalize(MEAN, STD) # Normalize with ImageNet stats
])
```

--- Convert OpenCV frame into model input ---

def preprocess(frame):

rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) **# Convert BGR → RGB**

from PIL import Image

img = Image.fromarray(rgb) **# Convert numpy array → PIL image**

return val_tf(img).unsqueeze(0).to(DEVICE) **# Apply transforms, add batch dimension, move to device**

--- Prediction function (no gradients needed) ---


```

@torch.no_grad()
def predict(x):
    out = model(x)                # Forward pass
    probs = torch.softmax(out, dim=1)    # Convert logits → probabilities
    conf, idx = torch.max(probs, dim=1)  # Get highest probability and its index
    return float(conf), classes[idx.item()] # Return confidence and class label

# --- MediaPipe setup for pose landmarks ---
mp_pose = mp.solutions.pose
mp_draw = mp.solutions.drawing_utils

# --- Helper: compute angle between three points (for feedback rules) ---
def angle(a, b, c):
    a, b, c = np.array(a), np.array(b), np.array(c)
    ba = a - b
    bc = c - b
    cos = np.dot(ba, bc) / (np.linalg.norm(ba) * np.linalg.norm(bc) + 1e-6)
    return np.degrees(np.arccos(np.clip(cos, -1, 1))) # Return angle in degrees

# --- Feedback rules based on predicted pose and landmarks ---
def feedback(label, lm, w, h):
    fb = []
    if lm is None: return fb # No landmarks → no feedback

    # Convert landmark to pixel coordinates
    def pt(l): return (int(l.x * w), int(l.y * h))

    # Extract key landmarks
    hip  = lm[mp_pose.PoseLandmark.LEFT_HIP.value]
    knee = lm[mp_pose.PoseLandmark.LEFT_KNEE.value]
    ankle = lm[mp_pose.PoseLandmark.LEFT_ANKLE.value]
    sho  = lm[mp_pose.PoseLandmark.LEFT_SHOULDER.value]
    rsho = lm[mp_pose.PoseLandmark.RIGHT_SHOULDER.value]

    # Compute angles/lines
    ang_knee = angle(pt(hip), pt(knee), pt(ankle))    # Knee bend angle
    shoulder_line = abs(pt(sho)[1] - pt(rsho)[1])    # Shoulder alignment (vertical difference)

    # Pose-specific feedback rules
    if "warrior" in label.lower():
        if ang_knee > 110: fb.append("Bend knee more.")
        if shoulder_line > 20: fb.append("Level shoulders.")
    if "goddess" in label.lower():
        if ang_knee < 90: fb.append("Lower squat.")
    if "plank" in label.lower():
        fb.append("Keep hips level.")
    if "tree" in label.lower():
        fb.append("Engage core for balance.")
    if "downdog" in label.lower():
        fb.append("Lift hips higher.")

    return fb

```

--- Webcam loop ---

cap = cv2.VideoCapture(2) **# Open camera (index 2 = DroidCam in your setup)**
buf = deque(maxlen=SMOOTHING_WINDOW) **# Buffer to smooth predictions across frames**

Use MediaPipe Pose with detection/tracking confidence thresholds

with mp_pose.Pose(min_detection_confidence=0.5, min_tracking_confidence=0.5) as pose:

while True:

ret, frame = cap.read()

if not ret: break

h, w = frame.shape[:2]

Detect landmarks

rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

res = pose.process(rgb)

lm = res.pose_landmarks.landmark if res.pose_landmarks else None

Draw skeleton if landmarks found

if res.pose_landmarks:

mp_draw.draw_landmarks(frame, res.pose_landmarks, mp_pose.POSE_CONNECTIONS)

Predict pose label

x = preprocess(frame)

conf, label = predict(x)

Apply confidence threshold

if conf < CONFIDENCE_THRESHOLD:

label = "Uncertain/Waiting"

Smooth predictions over last N frames

buf.append(label)

smooth = max(set(buf), key=buf.count)

Generate feedback based on smoothed label + landmarks

fb = feedback(smooth, lm, w, h)

Overlay text on frame

cv2.putText(frame, f"Pose: {smooth}", (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

cv2.putText(frame, f"Conf: {conf:.2f}", (20, 80), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255),

2)

Display feedback lines

y = 120

for line in fb:

cv2.putText(frame, line, (20, y), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)

y += 30

Show live window

cv2.imshow("Yoga Pose Recognition", frame)

if cv2.waitKey(1) & 0xFF == ord('q'): break **# Quit on 'q'**

--- Cleanup ---

```
cap.release()  
cv2.destroyAllWindows()
```